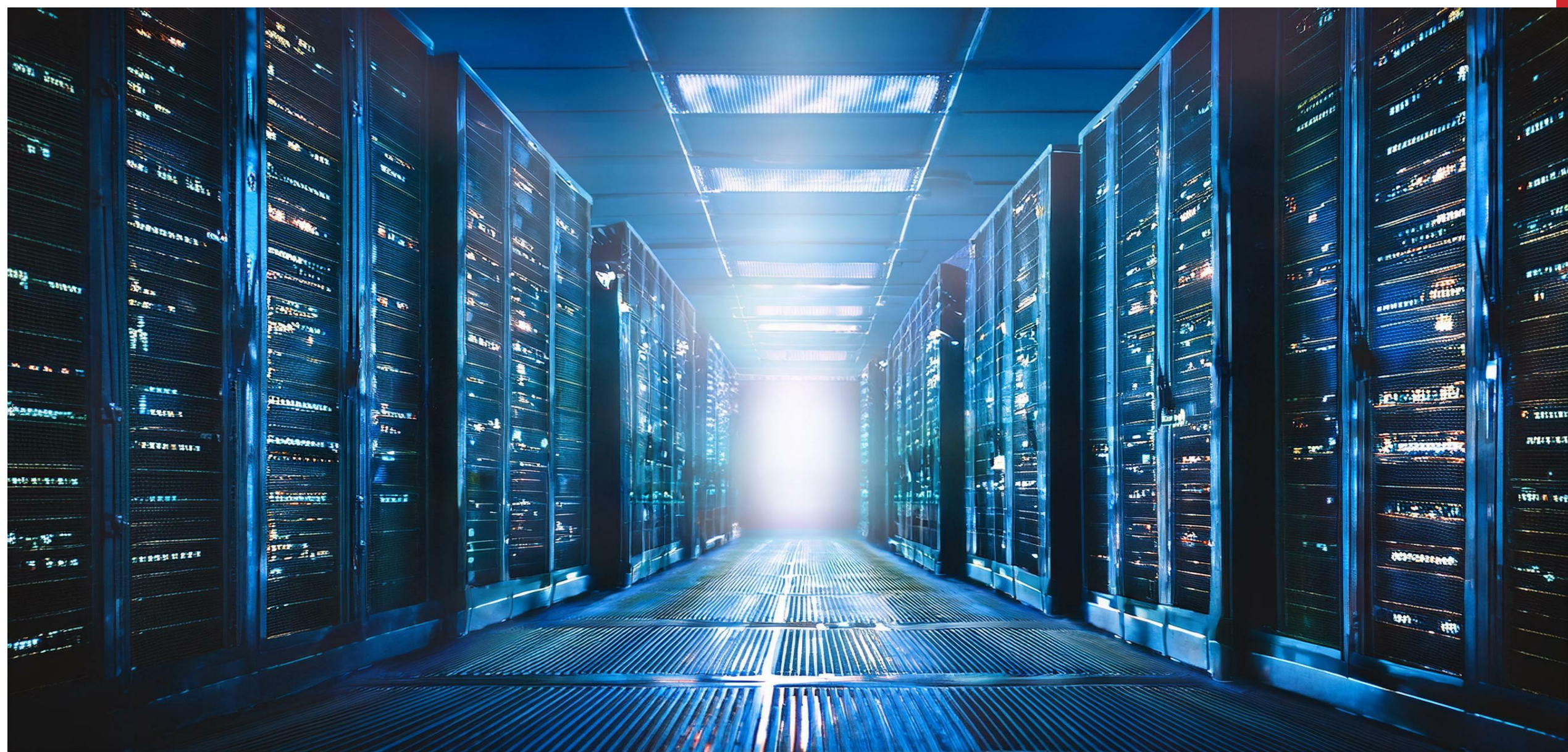


Cybersecurity in AI



WHAT IS CYBERSECURITY?

Cybersecurity is the practice of protecting systems, networks, and data from digital attacks, unauthorized access, and damage. Every organization that operates digitally depends on cybersecurity controls to stay operational and trustworthy.

- Confidentiality — keeping data private and accessible only to authorized users
- Integrity — ensuring data is accurate and has not been tampered with
- Availability — keeping systems and services running when needed
- Threat actors range from individual hackers to nation-state groups with sophisticated toolsets



THE CIA TRIAD

The CIA Triad is the foundational model for information security, organizing all security objectives into three properties.

Every security control you implement, from encryption to firewall rules, maps back to at least one of these three.

Property	Goal	Example Control
Confidentiality	Prevent unauthorized disclosure	Encryption, access control
Integrity	Prevent unauthorized modification	Hashing, audit logs
Availability	Ensure consistent access	Redundancy, backups

THE MODERN THREAT LANDSCAPE

Modern attackers target cloud infrastructure, identities, and APIs; not just on-premises servers. The attack surface has expanded dramatically as organizations moved workloads to cloud providers and adopted remote work.

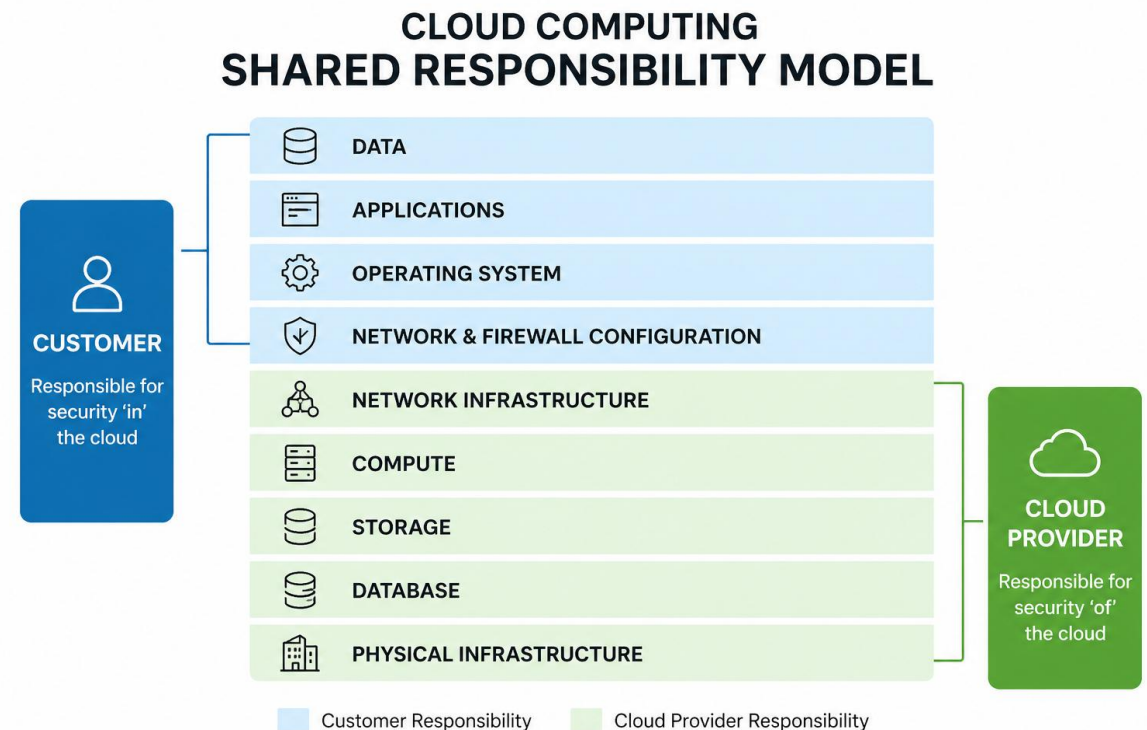
- Ransomware attacks have disrupted critical infrastructure in healthcare, energy, and logistics
- Credential theft is now the leading initial access technique across industries
- Supply chain attacks compromise software before it ever reaches the end user
- Misconfigurations in cloud accounts are among the most common causes of data breaches



CLOUD COMPUTING AND THE NEW PERIMETER

Traditional network security assumed a clear boundary between "inside" and "outside" the organization.

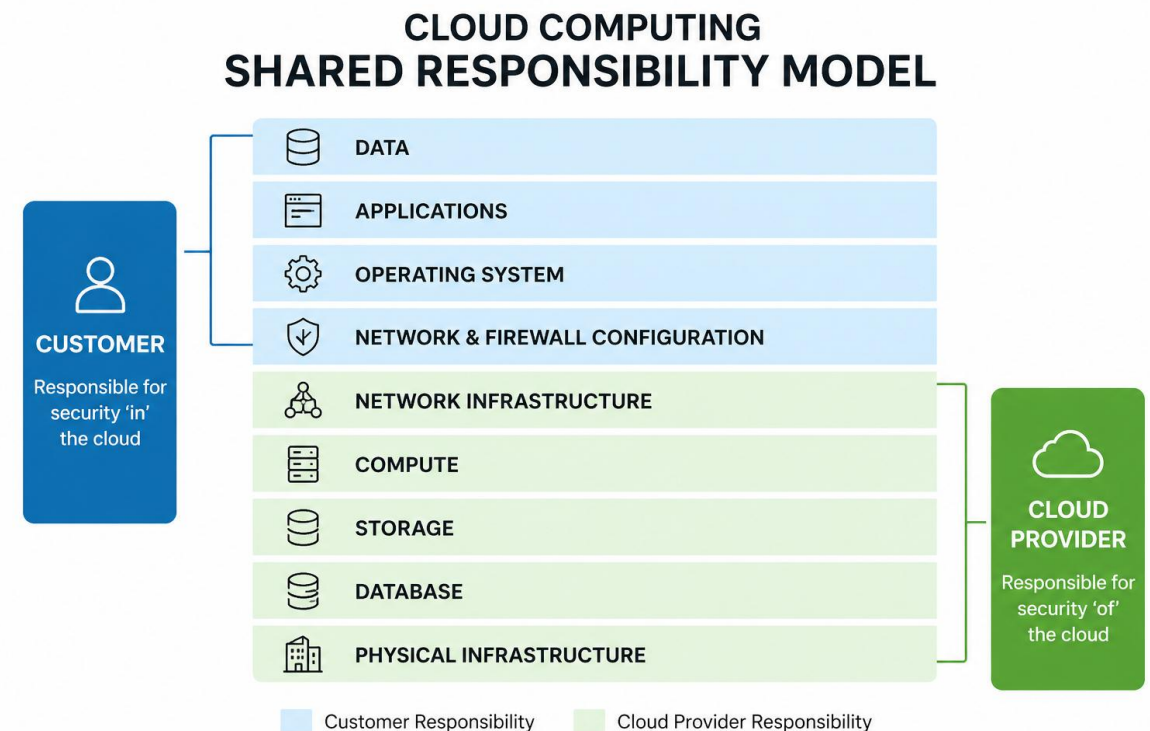
Cloud computing dissolved that perimeter. Resources are reachable from anywhere, and identity has become the new security boundary.



CLOUD COMPUTING AND THE NEW PERIMETER

The shared responsibility model defines the split between cloud provider and customer:

- Provider responsibility — physical infrastructure, hypervisor, and managed services
- Customer responsibility — OS configuration, access controls, data, and applications
- Misconfiguration of customer-controlled resources is the most common cloud security failure

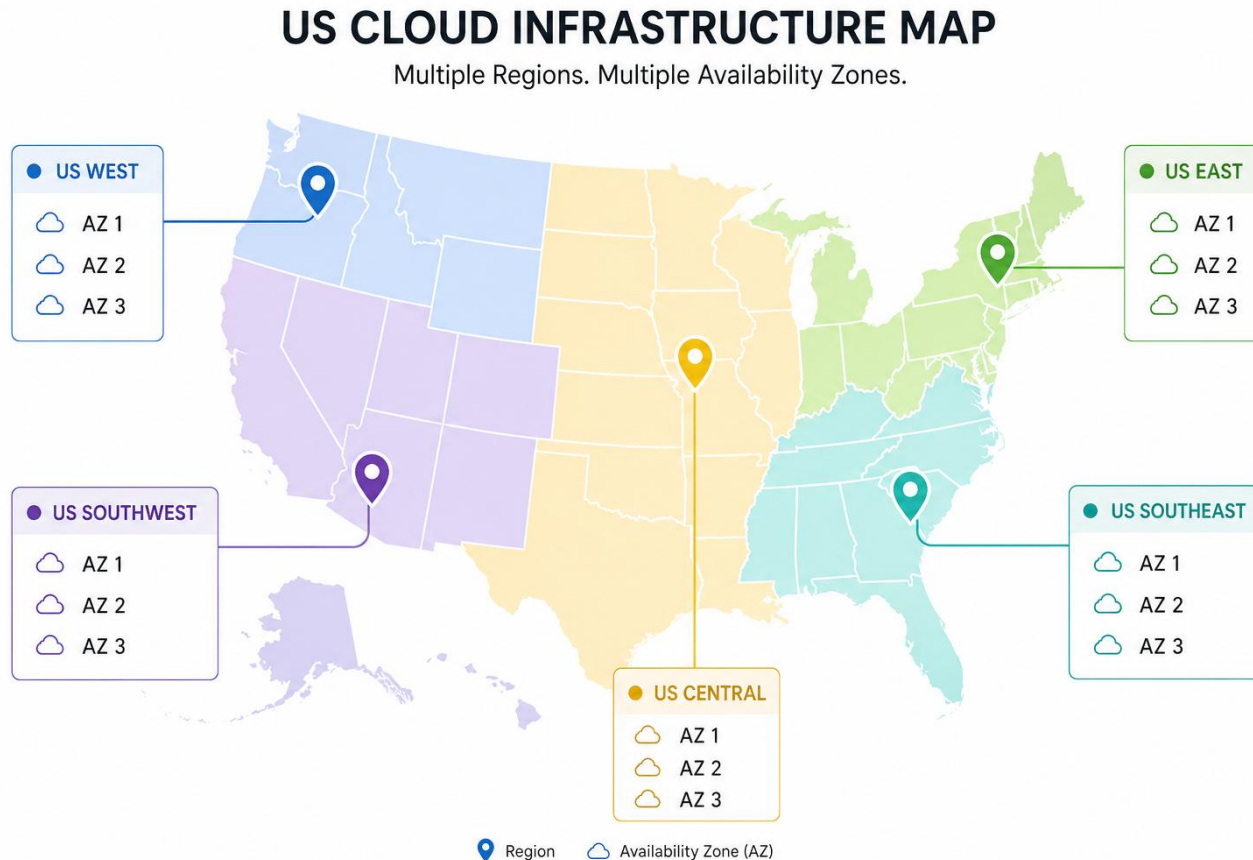


AWS AND CLOUD INFRASTRUCTURE

Amazon Web Services is the largest cloud provider, offering hundreds of services ranging from virtual machines to AI platforms and managed databases. Understanding its core building blocks is essential for cloud security work.



AWS AND CLOUD INFRASTRUCTURE



Region — a geographic cluster of data centers (e.g., us-east-1)

Availability Zone (AZ) — an isolated data center within a region

IAM — Identity and Access Management; controls who can do what

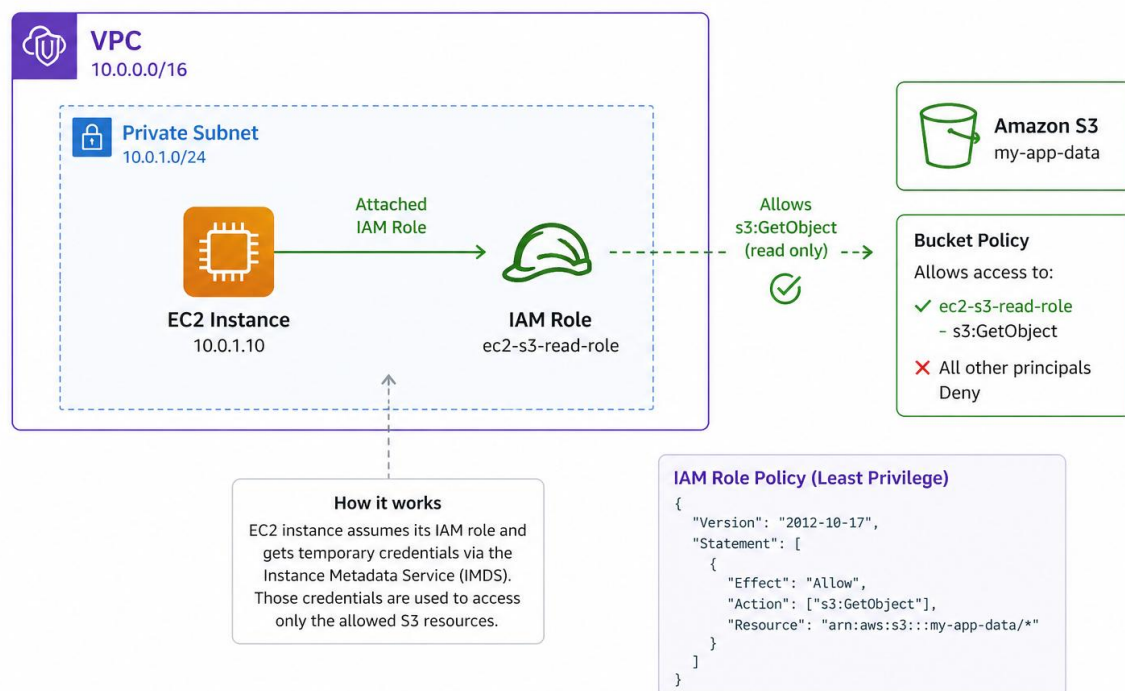
VPC — Virtual Private Cloud; an isolated network environment for your resources

EC2 — Elastic Compute Cloud; virtual machines you can launch and manage on demand

AWS AND CLOUD INFRASTRUCTURE

Least Privilege Access: EC2 with IAM Role accessing S3

EC2 instance gets temporary credentials from its IAM role to access only the required S3 bucket



Region — a geographic cluster of data centers (e.g., us-east-1)

Availability Zone (AZ) — an isolated data center within a region

IAM — Identity and Access Management; controls who can do what

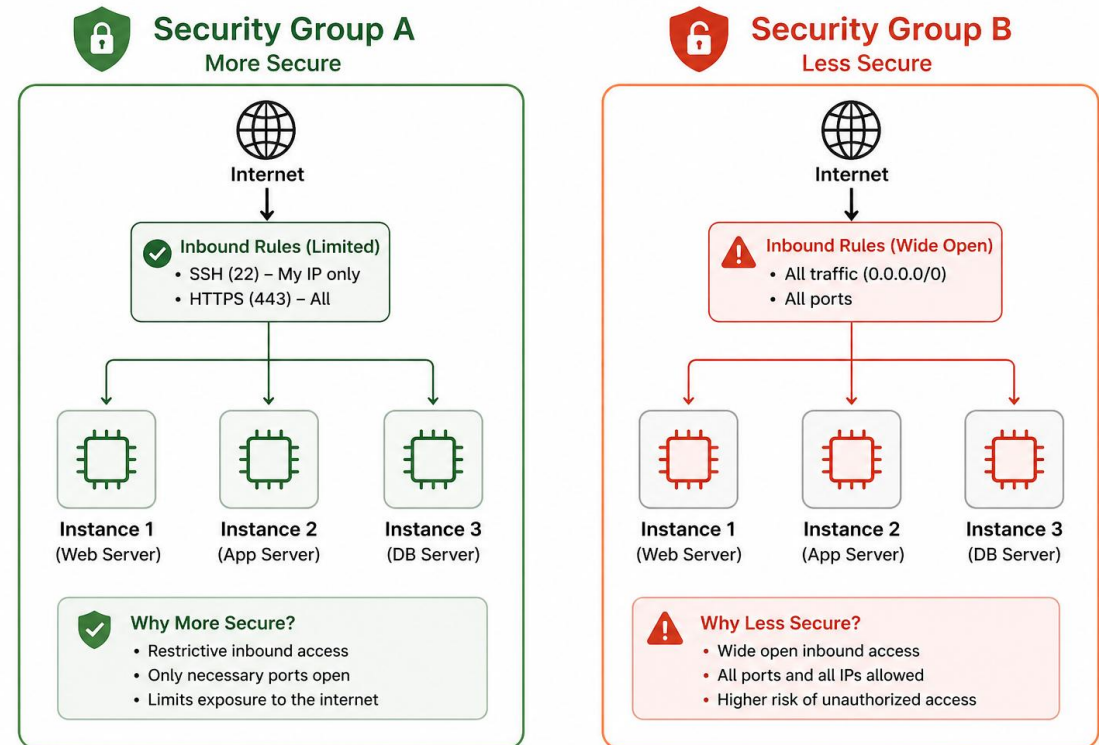
VPC — Virtual Private Cloud; an isolated network environment for your resources

EC2 — Elastic Compute Cloud; virtual machines you can launch and manage on demand

VIRTUAL MACHINES AND EC2 INSTANCES

An EC2 instance is a virtual machine running in AWS. You choose the operating system, instance type (CPU and RAM), networking configuration, and storage.

- AMI (Amazon Machine Image) — the launch template; includes the OS and base software
- Instance type — determines CPU and memory (e.g., t3.micro for lightweight lab workloads)
- Key pair — used for SSH access;
- Security group — a virtual firewall that controls inbound and outbound traffic



LAUNCHING EC2 INSTANCES SAFELY



Checklist for a safe EC2 launch:

- Restrict SSH access (port 22) to your IP. Never use 0.0.0.0/0 in production
- Use key-based authentication and disable password-based SSH login
- Choose a minimal, trusted AMIs. Avoid images with unknown pre-installed software
- Tag resources from the start so they are identifiable and auditable
- Do not attach AdministratorAccess IAM roles unless explicitly required

SSH — SECURE SHELL

SSH is a network protocol for encrypted communication with remote hosts. It replaced older plaintext protocols like Telnet and rlogin.

- Port 22 — default SSH port; heavily probed by automated internet scanners
- Encryption — all traffic is encrypted end-to-end after the initial key exchange
- Authentication options — password (weak) or public/private key pair (strong)
- SSH client — built into macOS and Linux; available via OpenSSH on Windows
- Once connected, you have a shell session with the same privileges as the logged-in user



SSH KEY-BASED AUTHENTICATION

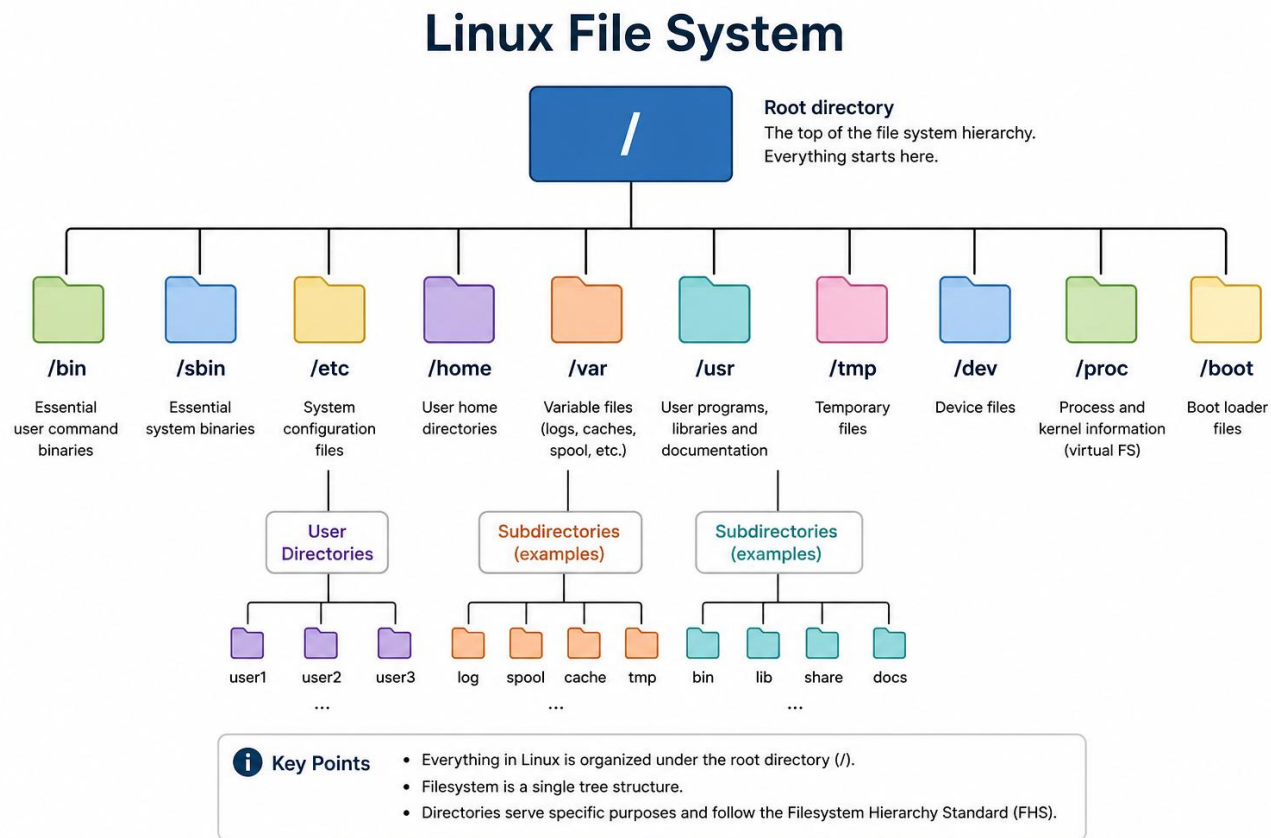
Key-based authentication replaces passwords with a cryptographic key pair. The private key never leaves your machine; the public key is installed on the server in `~/.ssh/authorized_keys`.

Two keys, one purpose:

- Private key (`.pem` / `id_rsa`) — stored securely on your local machine; treat it like a password
- Public key — stored on the server; mathematically linked to the private key
- Authentication succeeds only if the private key matches the public key on the server



NAVIGATING THE LINUX FILESYSTEM



Linux organizes everything under a single root directory /.

Understanding this hierarchy is essential for locating logs, configuration files, and security-relevant artifacts during an investigation.

LINUX FILE PERMISSIONS

Every file and directory in Linux has an owner, a group, and a set of permissions for three audiences: the owner, the group, and everyone else. Misconfigurations here are a frequent privilege escalation vector.

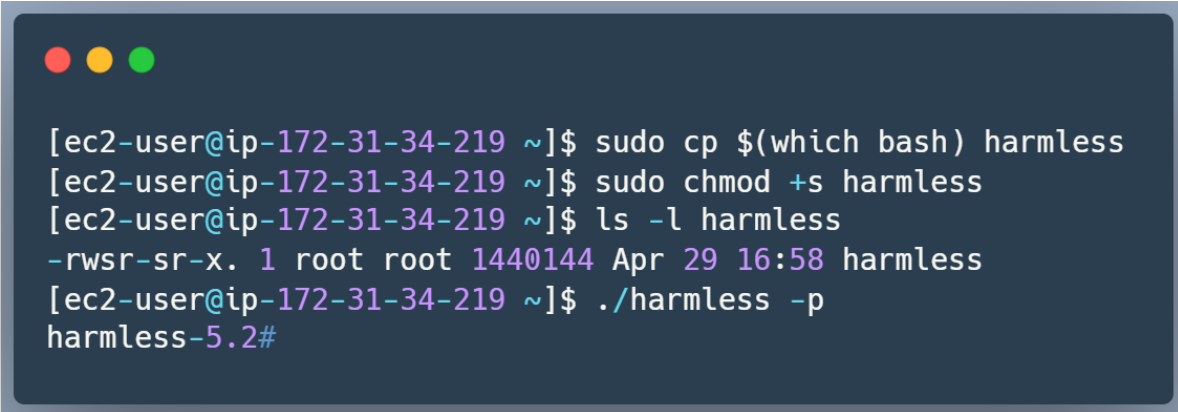
- Permission string breakdown — `rw-r--r--`:
- `chmod 600 key.pem` — restrict a private key to owner read/write only
- World-writable files (`777`) are a common misconfiguration that attackers exploit

Segment	Audience	Meaning
<code>rwX</code>	Owner	Read, write, execute
<code>r-X</code>	Group	Read and execute
<code>r--</code>	Others	Read only

SETUID BINARIES AND PRIVILEGE RISK

Normally, a program runs with the privileges of the user who launched it. A SetUID binary runs with the privileges of its owner.

- SetUID is intentional for tools like sudo, passwd, and ping. The risk comes from:
- Programs with SetUID that have known vulnerabilities, enabling instant privilege escalation
- Custom SetUID binaries placed by an attacker as a persistence technique

A terminal window with a dark blue background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows a series of commands and their outputs. The prompt is [ec2-user@ip-172-31-34-219 ~]. The commands are: sudo cp \$(which bash) harmless, sudo chmod +s harmless, ls -l harmless, and ./harmless -p. The outputs are: harmless, -rwsr-sr-x. 1 root root 1440144 Apr 29 16:58 harmless, and harmless-5.2#. The prompt changes to harmless-5.2# after the last command.

```
[ec2-user@ip-172-31-34-219 ~]$ sudo cp $(which bash) harmless
[ec2-user@ip-172-31-34-219 ~]$ sudo chmod +s harmless
[ec2-user@ip-172-31-34-219 ~]$ ls -l harmless
-rwsr-sr-x. 1 root root 1440144 Apr 29 16:58 harmless
[ec2-user@ip-172-31-34-219 ~]$ ./harmless -p
harmless-5.2#
```

SETUID BINARIES AND PRIVILEGE RISK

Find all SetUID binaries with:
`find / -perm -4000`
`2>/dev/null.`

Any unexpected entries in that list should be investigated immediately

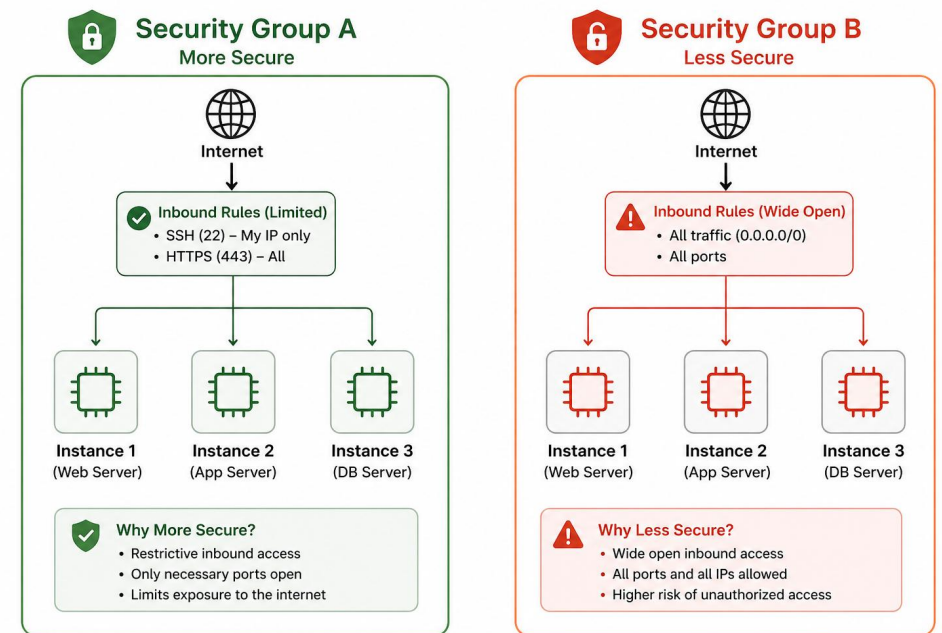


```
[ec2-user@ip-172-31-34-219 ~]$ sudo find . -perm -4000 \
-exec ls -l {} \; 2> /dev/null
-rwsr-sr-x. 1 root root 1440144 Apr 29 16:58 ./harmless
```


SECURITY GROUPS AS FIREWALLS

A security group is a stateful virtual firewall attached to an EC2 instance. It controls what traffic is allowed in (inbound rules) and out (outbound rules) at the instance level.

- Rules are allowlist-based — anything not explicitly allowed is denied
- Stateful — if you allow inbound traffic, the return response is automatically permitted
- Common mistake: SSH (0.0.0.0/0) left open after initial setup
- Each rule specifies: protocol, port range, and source/destination CIDR or security group
- Security groups are your first line of defense against remote exploitation of services



RUNNING AND SECURING A WEB SERVICE

A web server like nginx serves HTTP content over port 80. In a lab context, a running web service generates traffic you can observe, detect, and analyze in later labs.

- Open only the ports the service requires and block everything else in the security group
- Run the service as a non-root user whenever possible
- Review access logs (`/var/log/nginx/access.log`) for unusual request patterns
- Enable SELinux (Security Enhanced Linux) if possible to prevent other processes from accessing web server files



POP QUIZ: Why is SSH key-based authentication preferred over password authentication for accessing remote servers?

- A. Key-based authentication is faster because it skips the cryptographic handshake step
- B. Passwords are transmitted in plaintext over SSH, making them easy to intercept
- C. Key-based authentication eliminates the need for any network encryption
- D. The private key is never transmitted over the network, making brute-force attacks ineffective



POP QUIZ: Why is SSH key-based authentication preferred over password authentication for accessing remote servers?

- A. Key-based authentication is faster because it skips the cryptographic handshake step
- B. Passwords are transmitted in plaintext over SSH, making them easy to intercept
- C. Key-based authentication eliminates the need for any network encryption
- D. The private key is never transmitted over the network, making brute-force attacks ineffective**



POP QUIZ: Running `ls -l` on a file shows `-rwx-----`. Which of the following correctly describes this permission set?

- A. The owner can read, write, and execute; no one else has any access
- B. The owner can read and write; the group can execute
- C. Everyone on the system can read, write, and execute the file
- D. The file is read-only for all users, including the owner



POP QUIZ: Running `ls -l` on a file shows `-rwx-----`. Which of the following correctly describes this permission set?

- A. The owner can read, write, and execute; no one else has any access**
- B. The owner can read and write; the group can execute
- C. Everyone on the system can read, write, and execute the file
- D. The file is read-only for all users, including the owner



POP QUIZ: You run a routine audit and discover that a production EC2 instance has an inbound security group rule allowing TCP port 22 from `0.0.0.0/0`. What is the most appropriate immediate response?

- A. Leave it — SSH is encrypted, so open access poses no real risk
- B. Disable the SSH service on the instance to prevent any remote access
- C. Restrict the SSH rule to your organization's IP range or your current IP address
- D. Enable MFA on the AWS console account to compensate for the broad access rule



POP QUIZ: You run a routine audit and discover that a production EC2 instance has an inbound security group rule allowing TCP port 22 from `0.0.0.0/0`. What is the most appropriate immediate response?

- A. Leave it — SSH is encrypted, so open access poses no real risk
- B. Disable the SSH service on the instance to prevent any remote access
- C. Restrict the SSH rule to your organization's IP range or your current IP address**
- D. Enable MFA on the AWS console account to compensate for the broad access rule



LAB 1.1 — INSTANCE SETUP



THE SOC ANALYST ROLE

A Security Operations Center (SOC) analyst monitors, detects, and responds to security events in real time. The role requires combining technical knowledge with structured analytical thinking, often under time pressure.

- Tier 1 — triage incoming alerts, close false positives, escalate true positives
- Tier 2 — investigate escalated incidents, correlate events across data sources
- Tier 3 — threat hunting, advanced forensics, and proactive detection engineering
- Core tools: SIEM platforms, endpoint detection, network logs, and cloud audit trails
- The analyst's primary output is a decision: escalate, remediate, or close



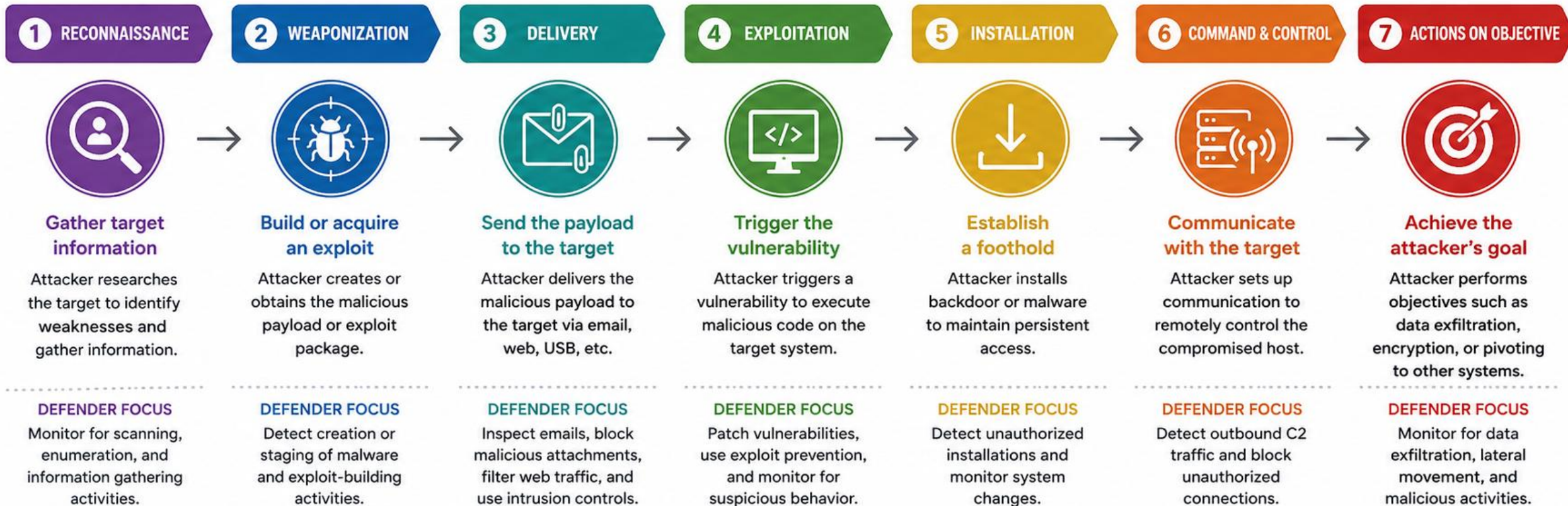
THE CYBER KILL CHAIN

The Cyber Kill Chain, developed by Lockheed Martin, describes the stages an attacker moves through from initial planning to achieving their objective. Understanding this model helps defenders identify where to detect and interrupt an attack before damage occurs.

1. Reconnaissance — gather target information
2. Weaponization — build or acquire an exploit
3. Delivery — send the payload to the target
4. Exploitation — trigger the vulnerability
5. Installation — establish a foothold
6. Command & Control — communicate with the compromised host
7. Actions on Objective — exfiltrate data, encrypt, or pivot

The Cyber Kill Chain™

The 7 Stages of an Attack Lifecycle



Key Takeaway: Attackers follow a sequence. Defenders can break the chain at any stage.

MITRE ATT&CK FRAMEWORK

ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) is a publicly available knowledge base of real-world attacker behaviors. It gives defenders and analysts a shared vocabulary to describe, detect, and respond to threats across every stage of an attack.

- Organized into Tactics (goals) and Techniques (how goals are achieved)
- Each technique has a unique ID (e.g., T1110 = Brute Force, T1548 = Privilege Escalation)
- Used to map detection rules to specific attacker behaviors
- Enables coverage gap analysis — which techniques have no current detection?

Reference: <https://attack.mitre.org>



BRUTE FORCE AND CREDENTIAL ATTACKS — T1110

Brute force attacks attempt to gain access by systematically trying many passwords or credential combinations against an authentication service. SSH is one of the most heavily targeted services for automated brute-force activity on the internet.

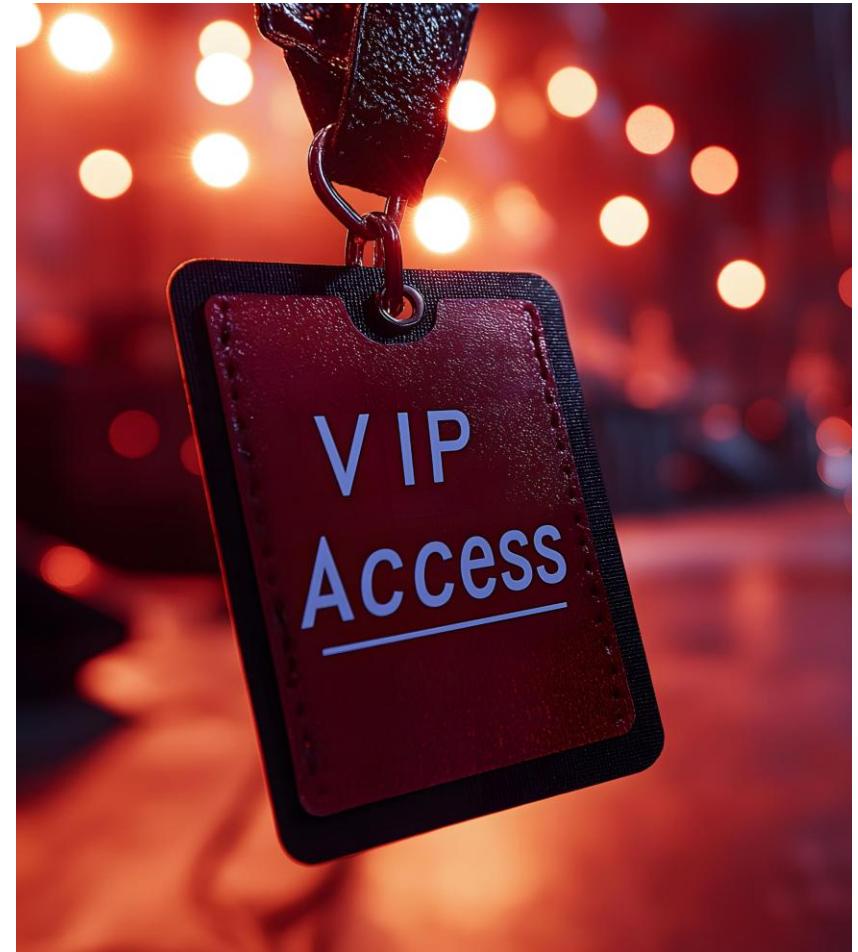
- T1110.001 — password guessing against a single known account
- T1110.003 — password spraying: one common password tried across many accounts
- Detection signals: high volume of failed auth events from a single source IP in a short window
- Log source on Linux: sshd journal entries and `/var/log/audit/audit.log`
- Effective mitigations: key-only SSH auth, fail2ban, rate limiting, non-default port



PRIVILEGE ESCALATION — T1548

Privilege escalation allows an attacker with limited access to gain higher privileges, typically root or Administrator. On Linux, this commonly exploits the sudo mechanism or SetUID programs.

- T1548 sub-techniques relevant to Linux:
- T1548.003 — sudo abuse; misconfigurations in /etc/sudoers grant unintended root access
- T1548.001 — SetUID and SetGID exploitation to execute code as a privileged owner
- Detection: sudo execution events in the audit log, especially unexpected commands
- Prevention: minimal sudoers entries, audit rules watching /etc/sudoers for modifications



WHY LOGS ARE THE FOUNDATION OF DETECTION

Without logs, an attacker can operate invisibly. Logs are the raw material feeding every detection rule, alert, and investigation.

Log coverage gaps are a deliberate attacker advantage.

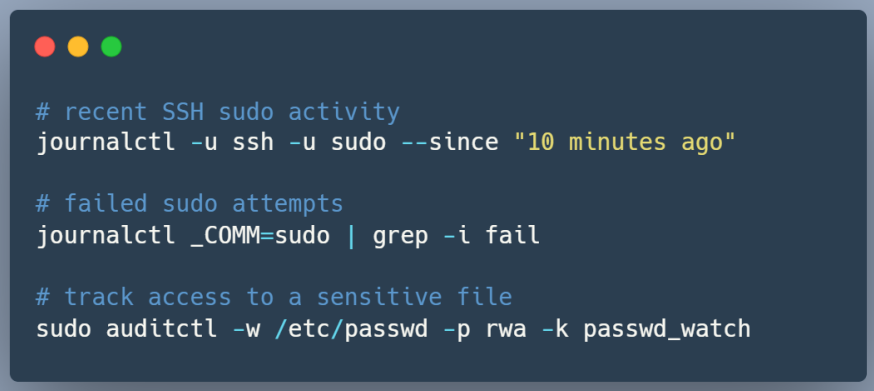
Logs capture authentication events, process execution, network connections, and file access

The challenge is not generating logs, but collecting, parsing, and retaining them usefully

In cloud environments, logs must be actively routed to a central store



LINUX LOGGING — JOURNALD AND AUDITD



```
# recent SSH sudo activity
journalctl -u ssh -u sudo --since "10 minutes ago"

# failed sudo attempts
journalctl _COMM=sudo | grep -i fail

# track access to a sensitive file
sudo auditctl -w /etc/passwd -p rwa -k passwd_watch
```

Two subsystems handle most security-relevant logging on modern Linux systems. Understanding both is essential for collecting the evidence needed in a SOC investigation.

journald (systemd-journald):

- Captures structured output from system services including sshd and sudo
- Queried with journalctl; supports time ranges, unit filters, and JSON output

auditd (Linux Audit Daemon):

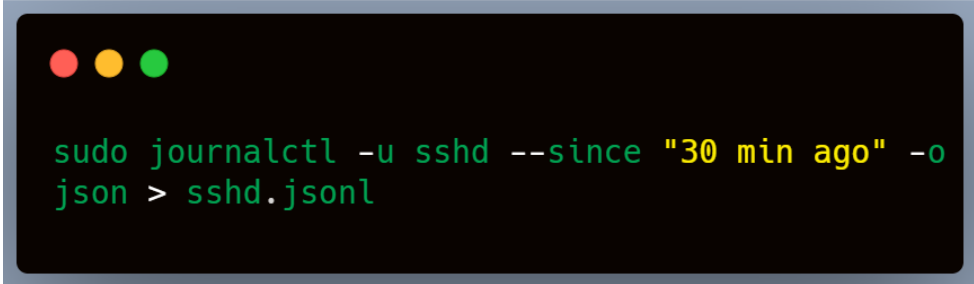
- Low-level kernel subsystem recording syscall activity and file access
- Configured via auditctl rules; logs to /var/log/audit/audit.log
- Captures privilege escalation attempts, sensitive file reads, and process execution

COLLECTING SSH EVIDENCE

SSH authentication events are among the most common signals analysts work with. Knowing where these events appear and what they look like is a prerequisite for writing detection logic.

Relevant event types:

- Failed password — wrong credential for a valid user
- Invalid user — authentication attempt for a username that does not exist
- Did not receive identification string — scanner or automated probe behavior
- Accepted publickey — successful key-based login (who logged in, and from where)

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays a command to collect SSH authentication logs from the journalctl database.

```
sudo journalctl -u sshd --since "30 min ago" -o  
json > sshd.jsonl
```


EVIDENCE AND THE ANALYST WORKFLOW

A SOC investigation is only as credible as the evidence collected. Analysts must gather, validate, and preserve evidence before drawing conclusions.

Structured evidence workflow:

- Capture — export raw log data with defined time boundaries
- Parse — extract structured fields (IP, user, timestamp, event type)
- Filter — isolate events relevant to the suspected behavior
- Correlate — connect events across sources (SSH logs, sudo logs, audit logs)
- Document — record findings with supporting evidence for handoff or escalation



RISK SCORING FUNDAMENTALS

Not all security events carry equal weight. Risk scoring assigns a numerical priority to entities based on the type and volume of suspicious activity they generated. This helps analysts focus effort on the highest-value targets first.



RISK SCORING FUNDAMENTALS

- Weighted signals — different event types score differently (e.g., ssh_burst_scan: 8 vs ssh_ident_probe: 2)
- Entity-based aggregation — scores accumulate per source IP or user, not per event
- Severity thresholds — high, medium, low tiers based on total accumulated score
- Explainability — every score must trace back to specific, reviewable evidence
- Scoring is a triage tool, not a verdict — the analyst validates the findings



PYTHON FOR SECURITY AUTOMATION

Python is the de facto scripting language for security automation. Its standard library covers file I/O, regular expressions, JSON parsing, and CSV output — everything needed to build a basic log analysis pipeline without installing any dependencies.

json module — parse JSON Lines log exports line by line

re module — extract IPs, usernames, and event types with compiled regex patterns

collections.defaultdict — aggregate counts per entity with minimal boilerplate

File I/O — write analyst-ready findings CSV files for handoff

Python produces repeatable, auditable results, unlike one-off manual grep workflows



TRIAGE NOTES AND DOCUMENTATION

A triage note is the analyst's written record of what was found, what it means, and what should happen next.

Entity — the IP address, username, or host under investigation

Severity — high, medium, or low, based on scored evidence

Evidence — the specific log lines or events that triggered the finding

Next pivot — what the analyst recommends as the next investigation step

Written immediately after scoring — context degrades quickly as analysts move between cases

TRIAGE NOTE EXAMPLE		 Written immediately after scoring Context degrades quickly as analysts move between cases.
 Entity The IP address, username, or host under investigation.	 Host: WS-23 (10.0.15.23) User: jsmith	
 Severity High, medium, or low, based on scored evidence.	HIGH Score: 85 / 100 Rationale: Multiple failed logins followed by successful login from unfamiliar IP and subsequent suspicious activity.	
 Evidence The specific log lines or events that triggered the finding.	2025-05-19 08:12:11 [AUTH] Failed login for user jsmith from 203.0.113.45 2025-05-19 08:12:23 [AUTH] Failed login for user jsmith from 203.0.113.45 2025-05-19 08:12:35 [AUTH] Failed login for user jsmith from 203.0.113.45 2025-05-19 08:13:02 [AUTH] Successful login for user jsmith from 203.0.113.45 2025-05-19 08:14:10 [PROCESS] User jsmith executed: powershell.exe -enc SQBFAF... 2025-05-19 08:15:47 [NETWORK] Outbound connection to 198.51.100.77:443	
 Next pivot What the analyst recommends as the next investigation step.	• Identify process tree and parent process for the PowerShell execution • Review outbound connection to 198.51.100.77 (reputation and WHOIS) • Check for data access or staging activity on WS-23 • Expand timeline ±1 hour for related events on WS-23 and 203.0.113.45	
 Analyst Notes User not traveling. No approved change activity. Likely compromise. Escalate for containment if more malicious activity confirmed. <i>Analyst: alex.smith Date/Time: 2025-05-19 08:25:14 UTC</i>		

POP QUIZ: Which of the following best describes MITRE ATT&CK technique T1110?

- A. An attacker installs a cron job to maintain persistence after a system reboot
- B. An attacker creates a new user account to establish a backdoor
- C. An attacker attempts to gain access by systematically trying many passwords against an authentication service
- D. An attacker escalates privileges by exploiting a misconfigured `sudo` rule



POP QUIZ: Which of the following best describes MITRE ATT&CK technique T1110?

- A. An attacker installs a cron job to maintain persistence after a system reboot
- B. An attacker creates a new user account to establish a backdoor
- C. An attacker attempts to gain access by systematically trying many passwords against an authentication service**
- D. An attacker escalates privileges by exploiting a misconfigured `sudo` rule



POP QUIZ: Your Python risk-scoring script reports the following for a source IP: entity: 203.0.113.47 severity: high score: 26 hit_summary: ssh_burst_scan:1 | ssh_failed_password:4 | ssh_invalid_user:2
What does this output tell you?

- A. The IP is a confirmed malicious address and should be blocked automatically
- B. The IP generated a high volume of SSH failures in a short window, including a burst scan pattern — the score warrants analyst investigation
- C. The script has a bug; scores above 20 indicate a parsing error
- D. The IP successfully authenticated and should be treated as a compromised account



POP QUIZ: Your Python risk-scoring script reports the following for a source IP: entity: 203.0.113.47 severity: high score: 26 hit_summary: ssh_burst_scan:1 | ssh_failed_password:4 | ssh_invalid_user:2
What does this output tell you?

- A. The IP is a confirmed malicious address and should be blocked automatically
- B. The IP generated a high volume of SSH failures in a short window, including a burst scan pattern — the score warrants analyst investigation**
- C. The script has a bug; scores above 20 indicate a parsing error
- D. The IP successfully authenticated and should be treated as a compromised account



POP QUIZ: A junior analyst says: "I can just read the raw log file — why do we need to run a scoring script?" Which response best explains the value of scored, ranked output over raw logs?

- A. Scoring scripts are required by compliance frameworks; raw log review is not permitted
- B. Raw logs cannot be read by humans and must always be pre-processed before review
- C. A raw log file may contain thousands of events; scoring aggregates them by entity and surfaces the highest-risk items first, reducing analysis time
- D. Scoring eliminates false positives, so analysts no longer need to validate findings manually



POP QUIZ: A junior analyst says: "I can just read the raw log file — why do we need to run a scoring script?" Which response best explains the value of scored, ranked output over raw logs?

- A. Scoring scripts are required by compliance frameworks; raw log review is not permitted
- B. Raw logs cannot be read by humans and must always be pre-processed before review
- C. A raw log file may contain thousands of events; scoring aggregates them by entity and surfaces the highest-risk items first, reducing analysis time**
- D. Scoring eliminates false positives, so analysts no longer need to validate findings manually



LAB 1.2 — SUSPICIOUS ACTIVITY SIMULATION AND RISK SCORING



AI IN CYBERSECURITY — OVERVIEW

AI tools are increasingly embedded in every layer of the security stack — from writing detection rules to triaging alerts and reviewing code.

For security practitioners, AI is most useful as an assistant that accelerates work while the analyst remains the decision-maker.



AI IN CYBERSECURITY — OVERVIEW

Code generation from drafting log parsers, scripts, and automation utilities

Log analysis — surfacing anomalies in large datasets

Policy review — identifying overpermissive configurations in IAM and cloud policies

Alert triage — generating initial hypotheses for analyst review

Threat intelligence — summarizing and correlating indicator data

The screenshot displays the 'Query definition' interface for Log Analytics. At the top, there's an 'Info' banner for 'Log Analytics' and a 'Start tailing' button. Below this, the 'Query scope' section includes a 'Log group name' dropdown, a search bar with the text '/soc-lab/secure', and a 'Clear all' button. The main query editor shows a KQL query: `1 fields @timestamp, @message`, `2+ filter @message like /sudo/`, `2 3 sort @timestamp desc`, `3 - limit 10000`, and `4+ limit 25`. Below the query editor, there's a navigation bar with 'Logs Insights QL', 'Query generator', 'Fields', 'Saved and sample queries', and 'Query commands'. The 'Query generator' tab is active, showing a 'Prompt' section with the text 'All logs containing sudo'. Below the prompt, there are buttons for 'Generate new query' and 'Refine existing query'. At the bottom, there's a row of buttons: 'Run query', 'Cancel', 'Save', 'Schedule query', and 'History'.

▼ Query definition

Info Log Analytics a unified observability platform for a smoother experience, now in preview mode. Click [here](#) to try it out!

30m 3h 1h [Compare \(Off\)](#) UTC timezone [Start tailing](#)

Query scope [Log groups](#) | [Property selector](#) ⓘ

Log group name Select up to 50 log groups Browse: [Log Groups](#) | [Facets](#) | [Lookup tables](#)

/soc-lab/secure Monitoring account 421377222628 [Clear all](#)

```
1 fields @timestamp, @message
2+ filter @message like /sudo/
2 3 sort @timestamp desc
3 - limit 10000
4+ limit 25
```

[Accept changes](#) ↶ ↷

[Logs Insights QL](#) [Query generator](#) [Fields](#) [Saved and sample queries](#) [Query commands](#)

Prompt Info 24/1000 ✕

All logs containing sudo

[Generate new query](#) [Refine existing query](#) ⚙️

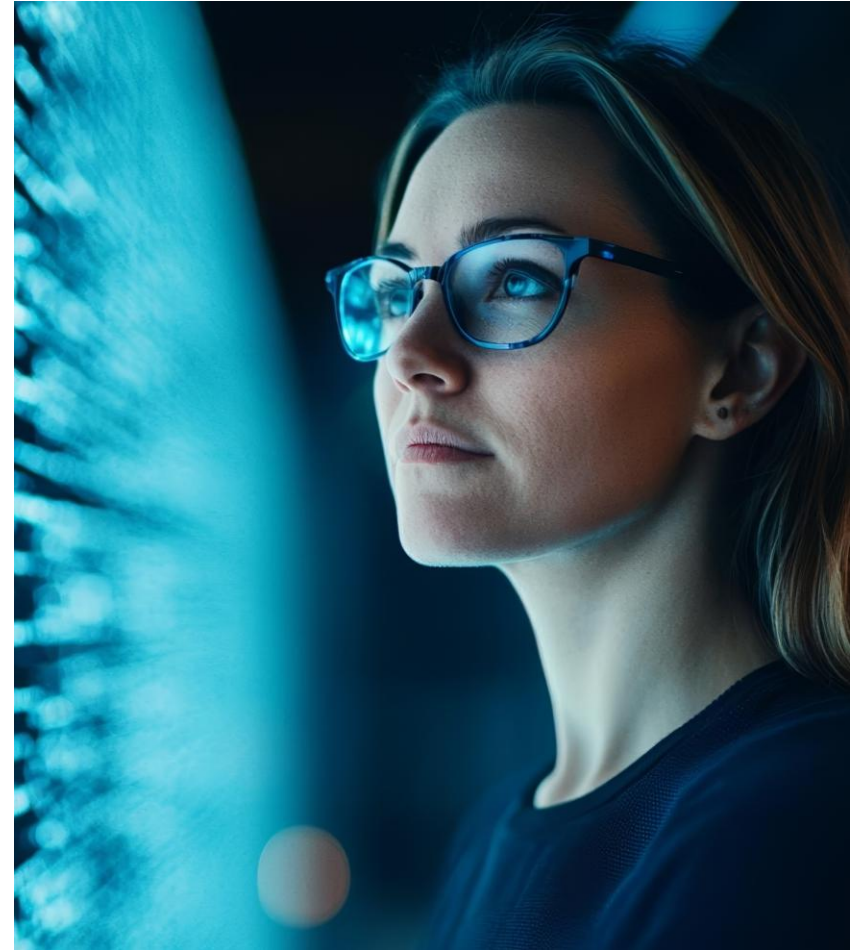
The selected log groups are used as context when generating queries.

[Run query](#) [Cancel](#) [Save](#) [Schedule query](#) ▼ [History](#)

AI CODING ASSISTANTS FOR SECURITY WORK

AI coding assistants like Amazon Q, Claude, and ChatGPT can produce functional Python, bash, and configuration code from plain English prompts. For security analysts, this dramatically lowers the barrier to building custom tooling without needing a software engineering background.

- Write a log parser for a custom or proprietary log format
- Generate a regex pattern for extracting fields from auth log entries
- Draft an IAM policy with least-privilege constraints
- Troubleshoot a broken Python script by pasting the error and asking for a fix
- Translate a detection idea into a working Logs Insights or Sigma rule



PROMPT ENGINEERING FOR SECURITY TASKS

The quality of AI output depends heavily on the quality of the prompt and domain knowledge of person building the prompt.

- Specify the environment — "Amazon Linux 2023, Python 3.11, no third-party libraries"
- Define the task precisely — include input format, expected output format, and edge cases
- Add constraints — "do not use eval(), do not write to paths outside the working directory"
- Request explanation — "explain each function before the code block" aids human review
- Iterate — treat first output as a draft; follow up with corrections rather than accepting it as final

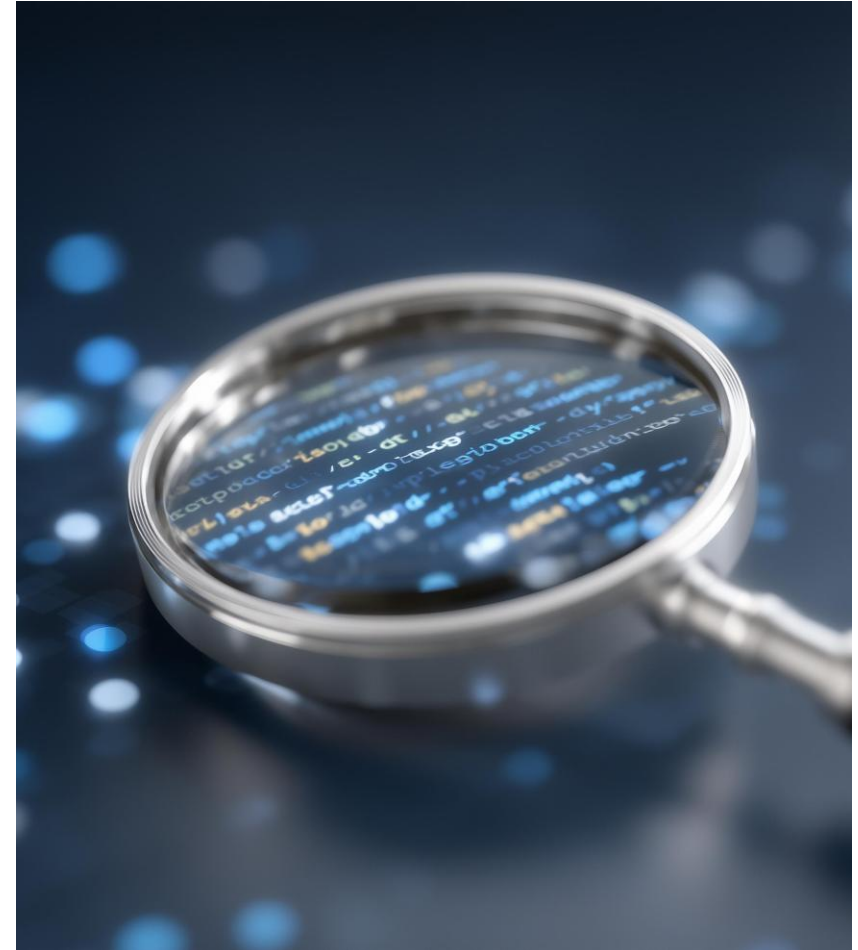


REVIEWING AND VALIDATING AI OUTPUT

AI-generated code can contain logic errors, insecure patterns, and outdated practices. Every piece of AI-generated code used in a security context must pass a human review step before execution.

A practical review checklist:

- Does the code do exactly what the prompt asked — nothing more, nothing less?
- Are there hardcoded credentials, tokens, or file paths that should not be there?
- Does it write to, delete, or modify files outside the expected working directory?
- Does it use `eval()`, `exec()`, or `subprocess` with `shell=True`?
- Does it handle errors gracefully, or will it silently fail and produce misleading output?



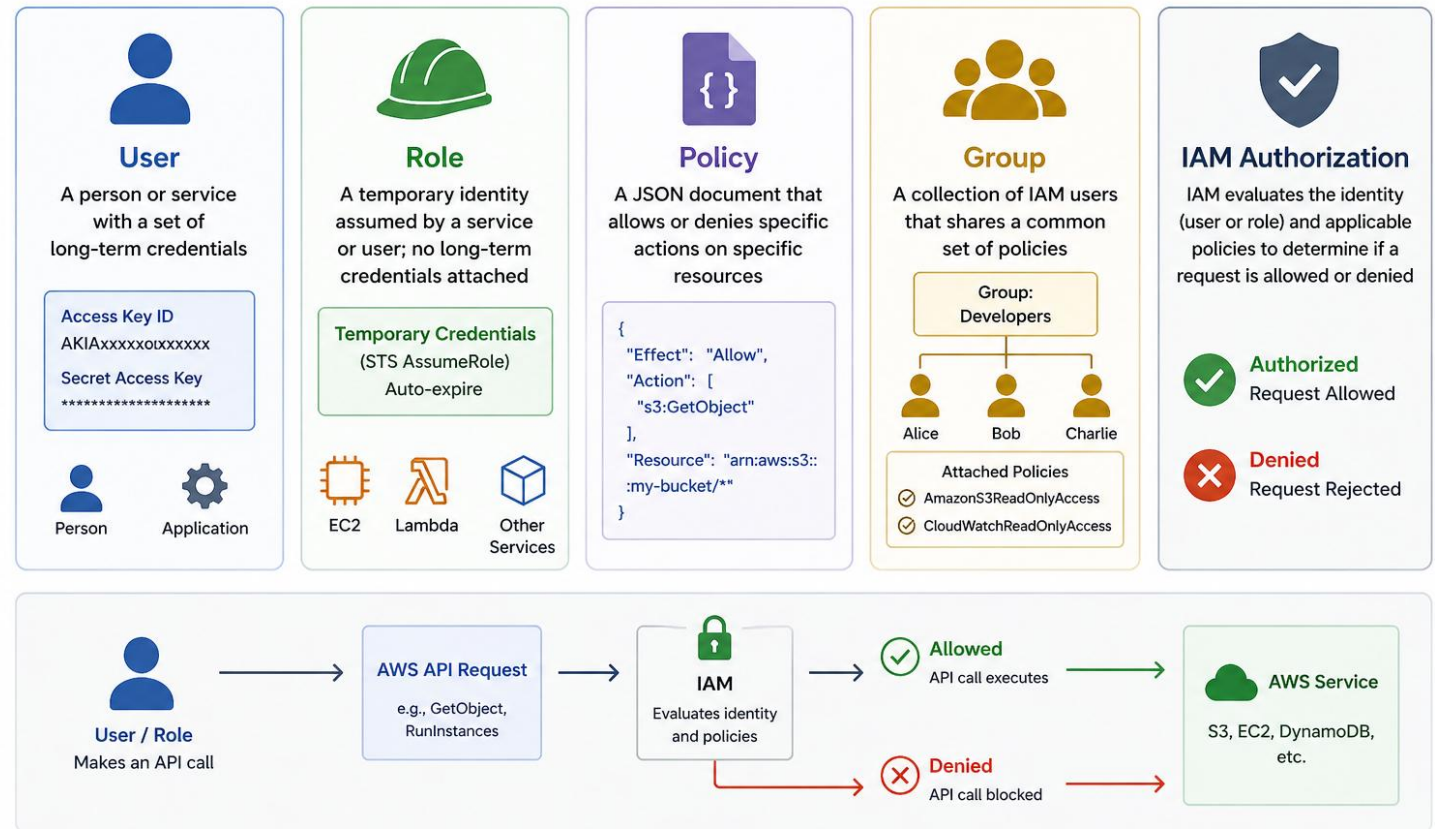
IAM FUNDAMENTALS

AWS Identity and Access Management (IAM) controls who can take what actions on which resources.

It is the central access control system for every AWS account and one of the most commonly misconfigured services in cloud environments.

Core IAM Concepts

Every AWS API call is authorized or rejected by IAM before it executes



IAM POLICY STRUCTURE

An IAM policy is a JSON document that grants or denies AWS API actions. Understanding policy structure is essential for both writing secure policies and identifying dangerous ones during a review.

Key fields:

Effect — Allow or Deny

Action — the specific API call(s) being controlled (e.g., s3:GetObject)

Resource — the ARN of the specific resource(s) the rule applies to

"Action": "*" combined with "Resource": "*" grants full access to everything — a critical finding

```
{  
  "Effect": "Allow",  
  "Action": ["s3:GetObject"],  
  "Resource": "arn:aws:s3::my-bucket/*"  
}
```

THE PRINCIPLE OF LEAST PRIVILEGE

Least privilege means granting the minimum permissions required for a task. It is one of the most important and most frequently violated security principles in cloud environments.

- A compromised role with Administrator Access can destroy an entire AWS account
- A compromised role scoped to s3:GetObject on one bucket has a limited blast radius
- Over permissive policies are easy to create and hard to roll back once services depend on them
- Practical approach:



THE PRINCIPLE OF LEAST PRIVILEGE

Practical approach:

1. Start with zero permissions and add only what is needed
2. Use IAM Access Analyzer to identify permissions granted but never actually used
3. Review policies quarterly — permissions accumulate over time and are rarely removed



LAMBDA EXECUTION ROLES

An AWS Lambda function runs code in response to events. Every Lambda function assumes an IAM execution role at runtime. The policies attached to that role determine what the function can do across your entire account.

Security considerations for Lambda roles:

- Scope logging permissions to exactly three actions: `CreateLogGroup`, `CreateLogStream`, `PutLogEvents`
- Use resource-level ARNs for S3 actions — never "Resource": `"*"`
- Never attach `AdministratorAccess` to a Lambda execution role
- Use a separate role per function — never share one role across all Lambdas

The execution role is a common lateral movement target after a Lambda code injection attack

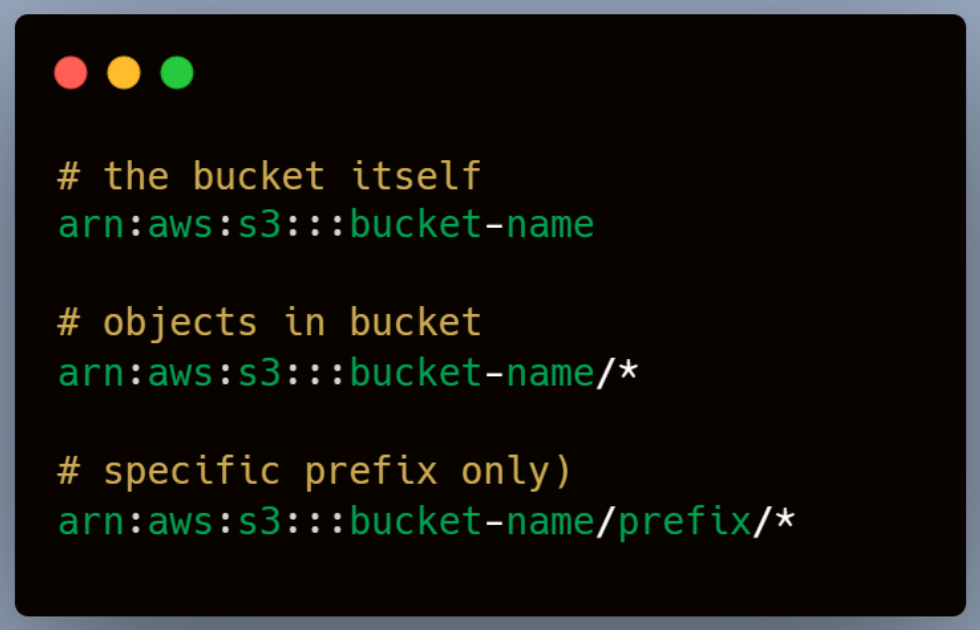
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:REGION:ACCOUNT_ID:log-group:/aws/lambda/FUNCTION_NAME:*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-secure-bucket/*"
    }
  ]
}
```

S3 RESOURCE-LEVEL ACCESS CONTROL

S3 permissions must be scoped to specific buckets and prefixes, not left open to all resources. Resource-level ARNs are the mechanism that enforces this scope in an IAM policy.

S3 ARN format:

- Granting `s3:GetObject` on `arn:aws:s3:::reports/2025/*` limits reads to that prefix only
- "Resource": "*" on any S3 action grants access to every bucket in the account
- `s3:::bucket-name` (missing `arn:aws:`) is an invalid ARN that can cause silent policy failures



```
# the bucket itself
arn:aws:s3:::bucket-name

# objects in bucket
arn:aws:s3:::bucket-name/*

# specific prefix only)
arn:aws:s3:::bucket-name/prefix/*
```

DETECTING IAM MISCONFIGURATIONS



IAM misconfigurations are difficult to spot by reading individual policies in isolation. A systematic approach that combines CLI enumeration with automated scanning is required to surface the most dangerous issues.

Common misconfigurations to hunt for:

- Policies with both "Action": "*" and "Resource": "*" (full wildcard)
- Users or roles with AdministratorAccess attached directly
- Unused IAM users with active access keys still in place
- Cross-account trust relationships not reviewed in the past 90 days
- Roles accessible by services with known code injection risk (e.g., Lambda, EC2 user data)

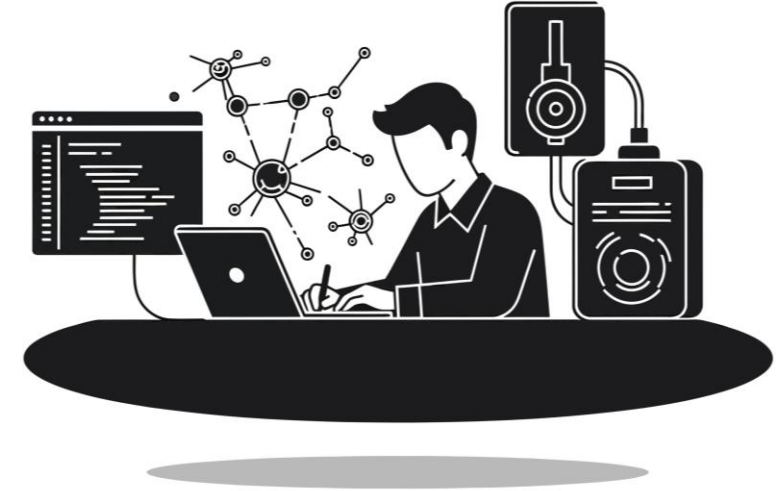
HUMAN-IN-THE-LOOP VALIDATION

Human oversight is not optional when using AI to generate security-sensitive code and configurations. The human in the loop is the safeguard that catches errors AI will not flag on its own.

Skipping any step converts AI-assisted work from an accelerator into a liability.

The loop in practice:

- Prompt — describe the task precisely, including constraints
- Review — read the output critically before acting on it
- Test — run in a safe, isolated environment; confirm the output matches expectations
- Validate — check security properties (no excess permissions, no risky patterns)
- Document — record what was generated, what was changed, and why



AI LIMITATIONS IN SECURITY CONTEXTS

AI tools have genuine blind spots that are especially consequential in security work.

- Outdated knowledge — training data has a cutoff; new CVEs and API changes may be unknown
- Confident hallucination — AI can state incorrect facts or invent non-existent APIs with high apparent confidence
- No execution context — AI cannot test whether code actually works in your environment
- No security accountability — AI will often generate unsafe patterns without warning
- Prompt injection risk — malicious content in log data passed to an AI tool can manipulate its output



POP QUIZ: A Lambda function reads objects from a single S3 bucket prefix (`reports/2025/`) and writes to CloudWatch Logs. Which IAM policy best follows the principle of least privilege?

- A. `"Action": "s3:*", "Resource": "*"`
- B. `"Action": ["s3:GetObject", "logs:*"], "Resource": "*"`
- C. `"Action": ["s3:GetObject", "logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"], "Resource": ["arn:aws:s3:::my-bucket/reports/2025/*", "arn:aws:logs:*"]`
- D. `"Action": "iam:PassRole", "Resource": "*"`



POP QUIZ: A Lambda function reads objects from a single S3 bucket prefix (`reports/2025/`) and writes to CloudWatch Logs. Which IAM policy best follows the principle of least privilege?

- A. `"Action": "s3:*", "Resource": "*"`
- B. `"Action": ["s3:GetObject", "logs:*"], "Resource": "*"`
- C. `"Action": ["s3:GetObject", "logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"], "Resource": ["arn:aws:s3::my-bucket/reports/2025/*", "arn:aws:logs:*"]`
- D. `"Action": "iam:PassRole", "Resource": "*"`



POP QUIZ: A developer asks an AI assistant to generate a Lambda IAM policy. The AI returns: { "Effect": "Allow", "Action": "*", "Resource": "*" } The developer deploys it without review because "the AI usually gets it right." What is the primary security risk?

- A. The policy will be rejected by AWS because wildcard actions are not permitted in IAM
- B. The Lambda function now has unrestricted access to every service and resource in the account, turning any code vulnerability into a full account compromise
- C. The policy grants read-only access because Lambda functions cannot perform write operations
- D. The risk is low because Lambda functions run in isolated execution environments



POP QUIZ: A developer asks an AI assistant to generate a Lambda IAM policy. The AI returns: { "Effect": "Allow", "Action": "*", "Resource": "*" } The developer deploys it without review because "the AI usually gets it right." What is the primary security risk?

- A. The policy will be rejected by AWS because wildcard actions are not permitted in IAM
- B. The Lambda function now has unrestricted access to every service and resource in the account, turning any code vulnerability into a full account compromise**
- C. The policy grants read-only access because Lambda functions cannot perform write operations
- D. The risk is low because Lambda functions run in isolated execution environments



POP QUIZ: Under which circumstance is it appropriate to deploy AI-generated security code directly to a production environment without human review?

- A. When the AI tool is specifically marketed as a "security-focused" assistant
- B. When the generated code is under 50 lines and therefore appears simple enough to be safe
- C. When the task seems low-risk, such as generating a log formatting utility
- D. None of the above — AI-generated code in a production security context always requires human review before deployment



POP QUIZ: Under which circumstance is it appropriate to deploy AI-generated security code directly to a production environment without human review?

- A. When the AI tool is specifically marketed as a "security-focused" assistant
- B. When the generated code is under 50 lines and therefore appears simple enough to be safe
- C. When the task seems low-risk, such as generating a log formatting utility
- D. None of the above — AI-generated code in a production security context always requires human review before deployment**



LAB 1.3 — AI-ASSISTED SECURITY SCRIPTING AND IAM REVIEW



CLOUD SECURITY POSTURE MANAGEMENT

Cloud Security Posture Management (CSPM) is the continuous process of identifying, assessing, and remediating misconfigurations across cloud environments.



CLOUD SECURITY POSTURE MANAGEMENT

- CSPM checks run continuously against your live cloud account configuration
- Findings are mapped to compliance frameworks: CIS, NIST, PCI-DSS, and others
- Unlike vulnerability scanners, CSPM works through cloud APIs — no network access or agents required
- AWS-native CSPM: Security Hub combined with AWS Config, no additional tooling needed
- The most common CSPM findings: open ports, over permissive IAM, unencrypted storage, missing MFA



AWS SECURITY SERVICES OVERVIEW

AWS provides a suite of native security services covering detection, posture management, access analysis, and audit logging.

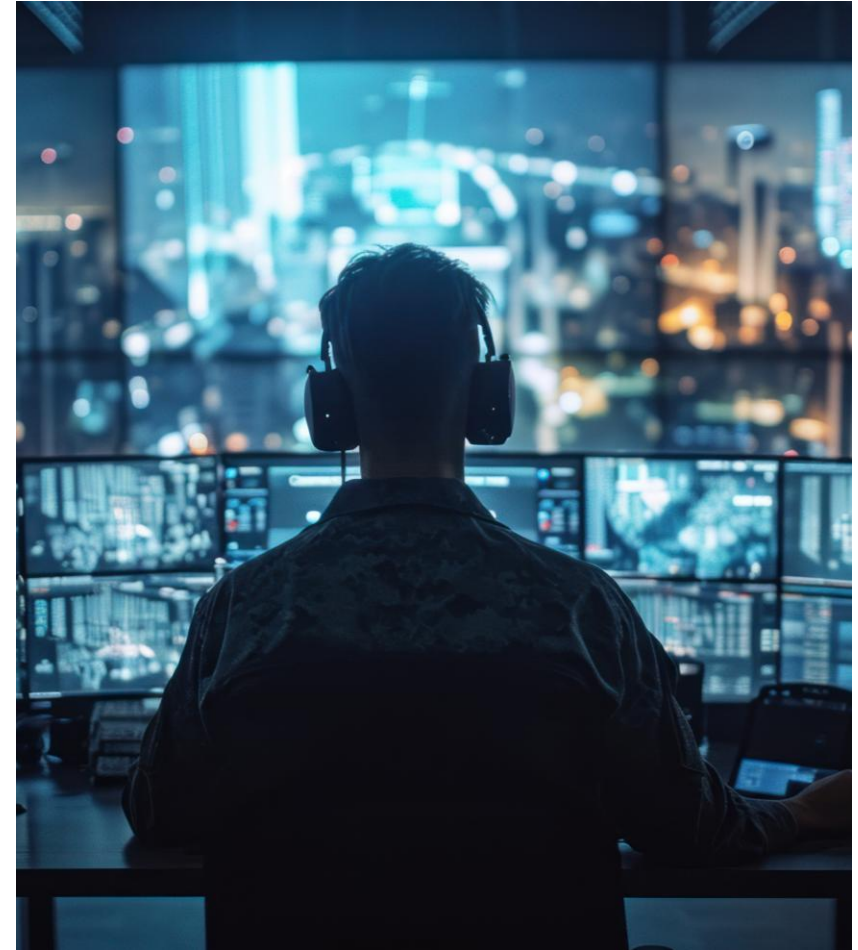
Each service addresses a different layer of the security stack. Used together, they provide broad visibility.

Service	Primary Function
Security Hub	Aggregates findings from multiple services into a central dashboard
GuardDuty	Threat detection via log analysis and behavioral ML
IAM Access Analyzer	Identifies resources accessible from outside the account
CloudTrail	Audit log of every API call made in the account
AWS Config	Tracks resource configuration changes over time
Inspector	Vulnerability scanning for EC2 and container workloads

AWS SECURITY HUB

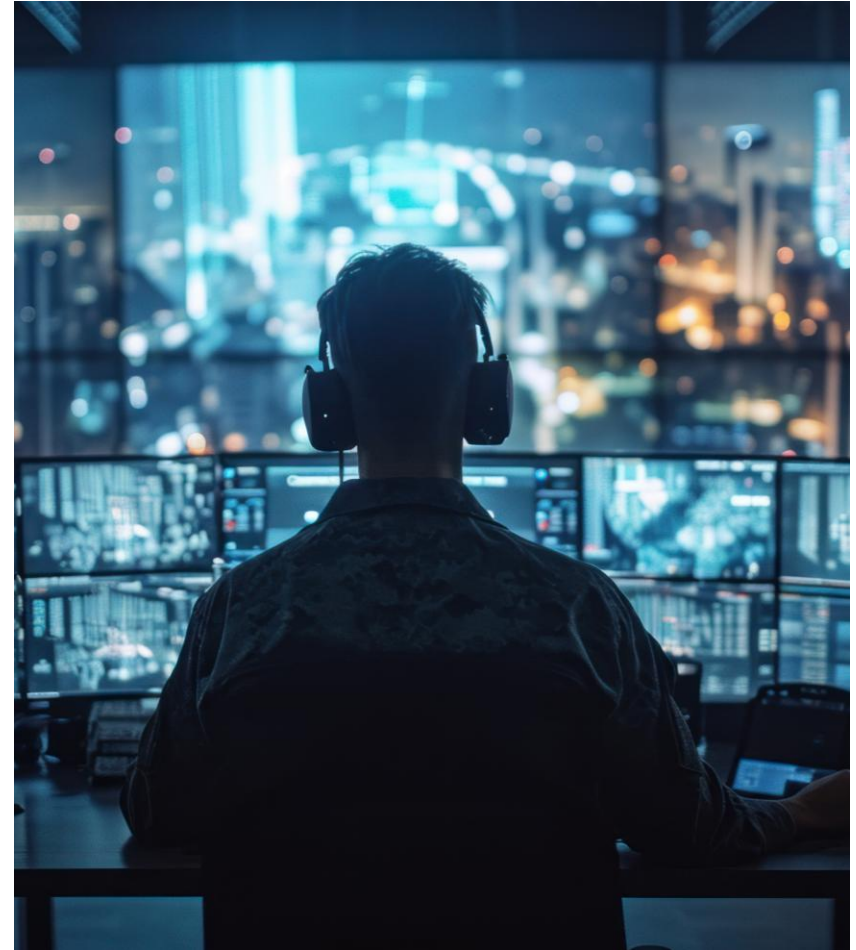
Security Hub is a central aggregation point for security findings across your AWS account.

It continuously evaluates resources against industry security standards and collects findings from integrated AWS services and third-party tools.



AWS SECURITY HUB

- Standards supported: CIS AWS Foundations Benchmark, AWS Foundational Security Best Practices (FSBP), PCI-DSS
- Findings carry a severity rating: Critical, High, Medium, or Low
- Aggregates findings from GuardDuty, Inspector, Macie, and IAM Access Analyzer automatically
- Provides an overall Security Score representing the percentage of controls currently passing
- Does not perform remediation, it surfaces findings for analyst or engineer action



AMAZON GUARDDUTY

GuardDuty is AWS's managed threat detection service. It continuously analyzes CloudTrail logs, VPC Flow Logs, and DNS query logs to identify malicious or unexpected behavior. Without requiring agents, network taps, or manual configuration.

GuardDuty detects:

- Reconnaissance — unusual API enumeration, port scanning from EC2 instances
- Credential compromise — API calls from Tor exit nodes or unexpected geographic locations
- Cryptomining — EC2 network traffic to known mining pools
- Policy violations — root credential use, public S3 bucket modifications
- Findings are classified as Low, Medium, or High with a description and recommended action



ROOT ACCOUNT SECURITY AND MFA

The AWS root account has unrestricted access to every service and resource in the account, including the ability to close the account entirely. It should never be used for day-to-day operations.

Root account security requirements:

- Enable MFA immediately — hardware security key preferred; an authenticator app is the minimum
- Do not create access keys for the root account — they cannot be scoped and should never exist
- Reserve root access for the specific tasks that require it (e.g., changing account contact details or payment information)



IAM CREDENTIAL HYGIENE

IAM credential hygiene is the ongoing practice of keeping access keys, user accounts, and permissions in a minimal, audited state. Stale and unused credentials are a significant exposure vector. Attackers actively search for forgotten access keys in code repositories and configuration files.

Regular hygiene tasks:

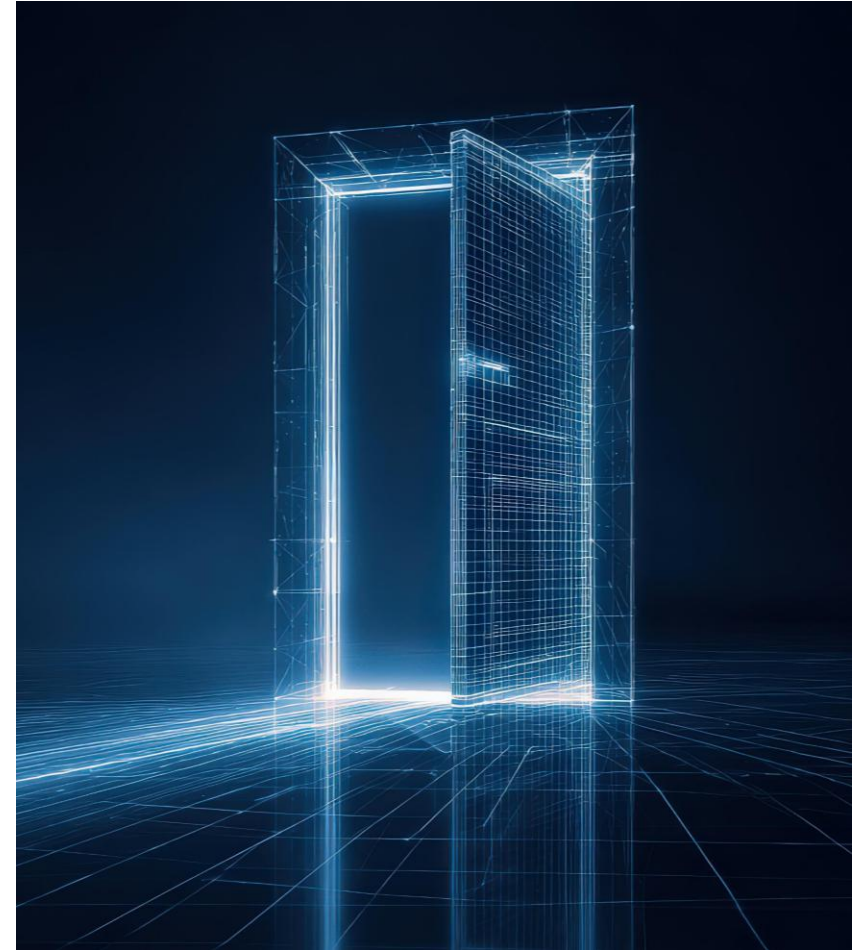
- Generate and review the IAM Credential Report regularly (aws iam get-credential-report)
- Rotate or delete access keys older than 90 days
- Disable or delete IAM users who have not logged in for 90 or more days
- Enforce MFA for all IAM users with console access
- Use IAM roles instead of long-term access keys wherever the service supports it



NETWORK EXPOSURE — SECURITY GROUPS

An internet-exposed port is an open door. Every port reachable from 0.0.0.0/0 is discoverable by automated scanners within minutes of an instance going live. Attackers run continuous internet-wide scans.

Correct configuration: scope each port to the minimum required source range; use AWS Systems Manager Session Manager to eliminate port 22 entirely.



NETWORK EXPOSURE — SECURITY GROUPS

High-risk security group misconfigurations:

- SSH (port 22) open to 0.0.0.0/0 — brute-force and credential-spray target
- RDP (port 3389) open to 0.0.0.0/0 — extremely high-value target for ransomware groups
- Database ports (3306, 5432, 27017) open to the internet
- Administrative panels (8080, 8443) exposed to public addresses



S3 BUCKET SECURITY

S3 is one of the most commonly misconfigured services in AWS. Publicly accessible S3 buckets have been the source of some of the largest data breaches in cloud history.

S3 security controls to verify:

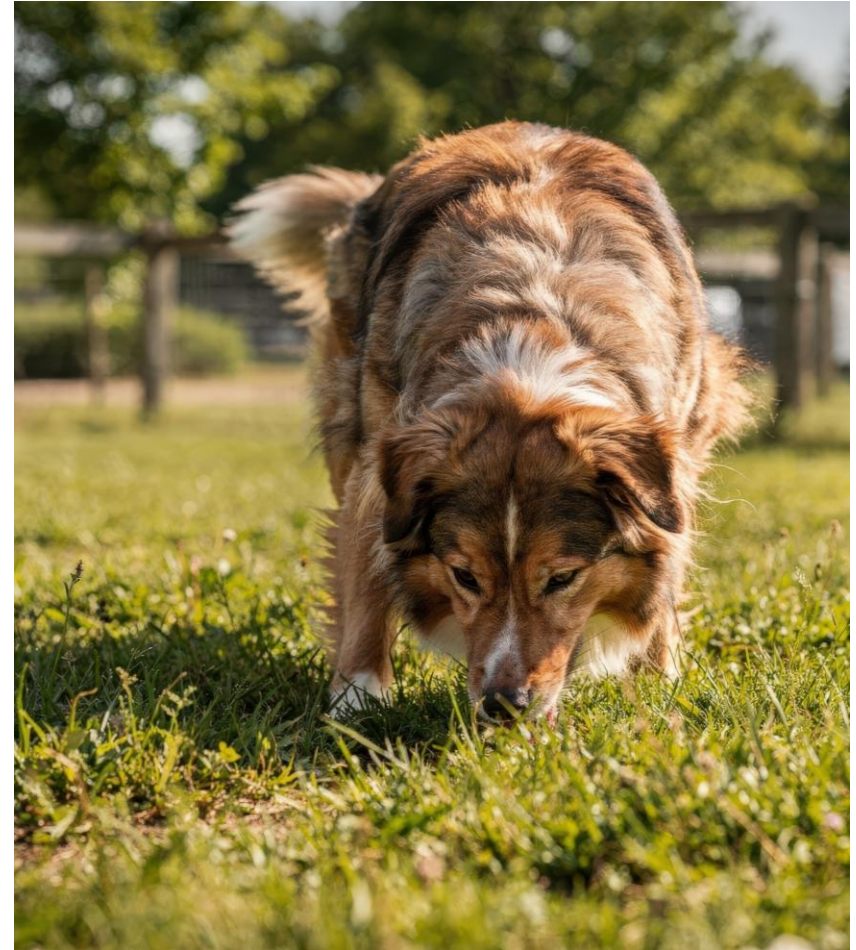
- Block Public Access — a four-flag setting preventing public ACLs and policies; enable on every bucket
- Bucket policy — review for "Principal": "*" which grants public read or write access
- ACLs — legacy mechanism; should be disabled on all new buckets
- Server-side encryption — verify all objects are encrypted at rest
- Security Hub S3.1 flags any bucket with Block Public Access disabled as a finding



AWS CLOUDTRAIL — AUDIT LOGGING

CloudTrail records every AWS API call made in your account, who made it, from where, what was requested, and what the response was.

It is the foundational audit log for cloud security investigations and forensics.



FINDING PRIORITIZATION AND SEVERITY

Not all security findings require the same urgency. A structured prioritization framework ensures the most dangerous issues are addressed before lower-risk items consume analyst time and attention.

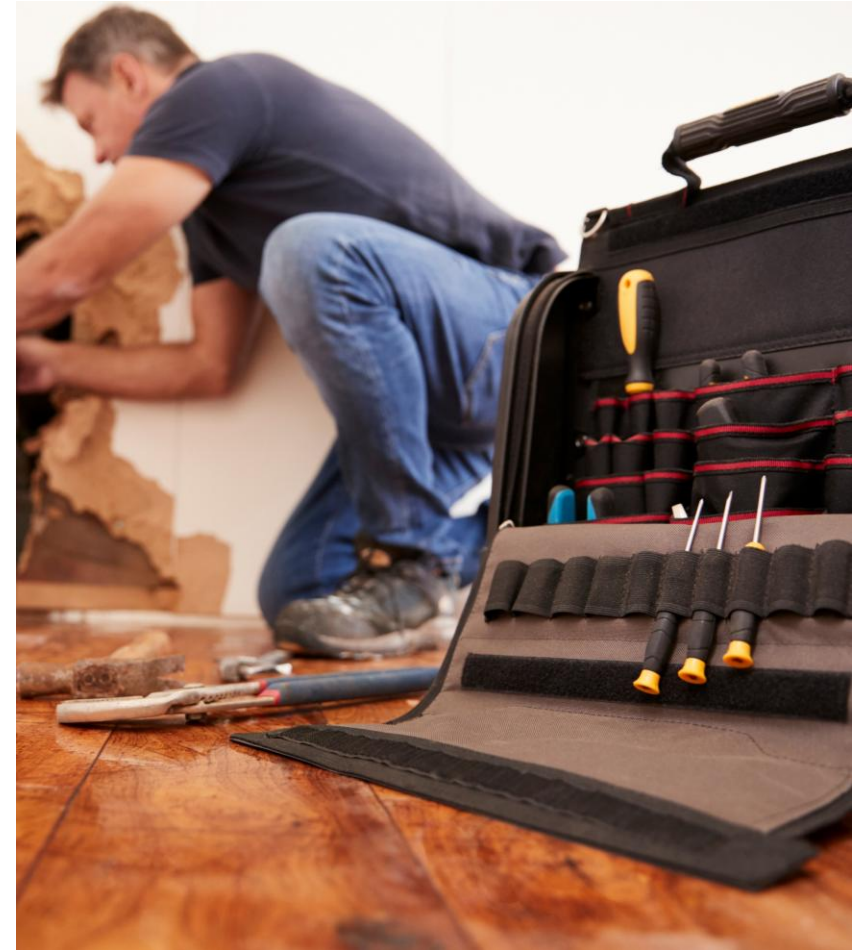
Always address Critical findings before moving to High. Internet-facing resource exposure almost always outranks identity or logging gaps.

Priority	Criteria	Example
Critical	Active exploitation risk, identity breach	Root access keys active
High	Internet exposure of critical services	SSH open to 0.0.0.0/0
Medium	Configuration gap that increases risk	MFA not enforced for IAM users
Low	Best-practice deviation, minimal direct risk	Unused IAM group with no members

CONTROLLED REMEDIATION PRINCIPLES

Remediation in a cloud environment must be planned and reversible. Hasty changes in production can break running services or create new exposures if they are not carefully scoped and validated.

Never use broad "fix everything at once" approaches in production. Isolate each remediation step.



CONTROLLED REMEDIATION PRINCIPLES

Safe remediation workflow:

- Document the current state before making any change
- Scope the change to the minimum required action
- Test — verify the resource still functions correctly after the change
- Verify — confirm the finding is resolved in Security Hub or by re-running the CLI check
- Record — log what changed, when, and who authorized it



CONTINUOUS COMPLIANCE MONITORING



A one-time security audit tells you the posture at a single point in time. Continuous monitoring detects when configuration drift reintroduces a finding that was previously resolved, which is far more common than most teams expect.

Continuous monitoring in AWS:

- AWS Config rules — evaluate resource configurations against defined policies on every detected change
- Security Hub — re-evaluates findings automatically and marks them resolved when the condition clears
- GuardDuty — runs continuously with no manual refresh or re-scan needed
- Set up Security Hub cross-account and cross-region aggregation for a single view across the organization

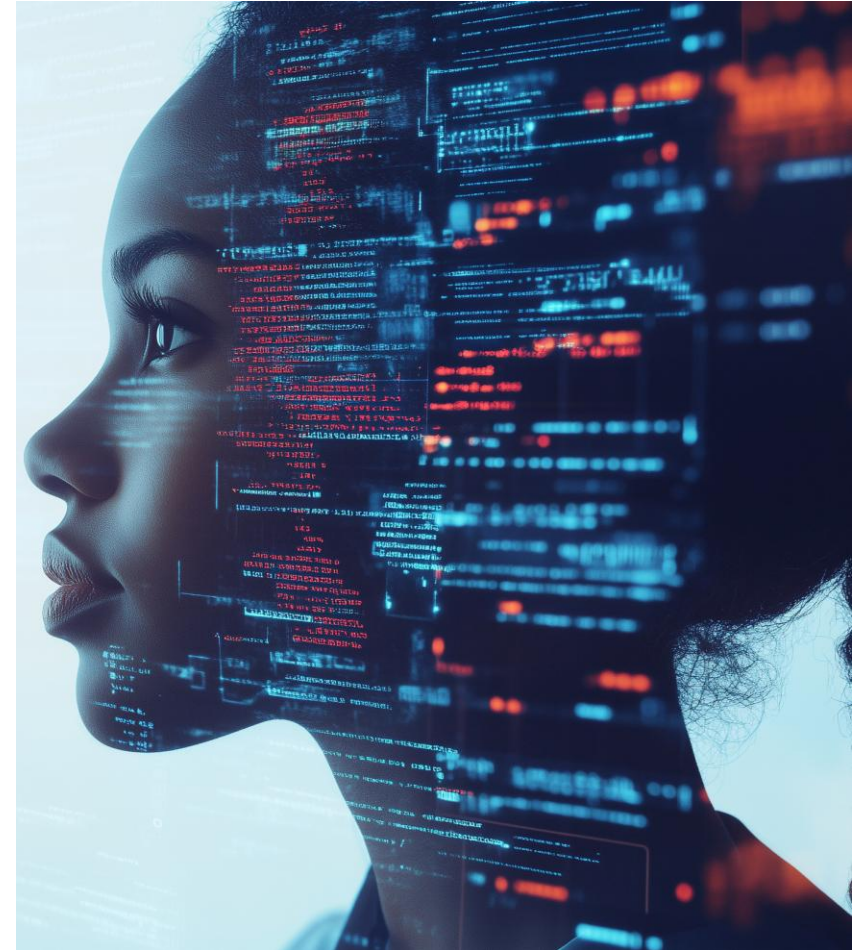
BUILDING A SECURITY FINDINGS BASELINE

A security findings baseline is a structured snapshot of the current state of your environment. It provides a reference point to measure improvement, demonstrate compliance, and communicate risk clearly to stakeholders.

Baseline components:

- Finding list — severity, title, resource type, resource ID, and current status for each finding
- Finding count by severity — Critical, High, Medium, and Low totals at a glance
- Remediation status — which items are open, in-progress, or resolved
- Stored as a structured file (CSV or JSON), not a screenshot — parseable, diffable, and auditable over time

A baseline produced today becomes the comparison point for every future assessment



POP QUIZ: What is the primary function of AWS Security Hub?

- A. It provides real-time threat detection by analyzing CloudTrail and VPC Flow Logs using machine learning
- B. It aggregates security findings from multiple AWS services and third-party tools into a central dashboard and evaluates resources against compliance standards
- C. It automatically remediates security findings without requiring analyst involvement
- D. It stores all security-related logs for seven years to meet compliance retention requirements



POP QUIZ: What is the primary function of AWS Security Hub?

- A. It provides real-time threat detection by analyzing CloudTrail and VPC Flow Logs using machine learning
- B. It aggregates security findings from multiple AWS services and third-party tools into a central dashboard and evaluates resources against compliance standards**
- C. It automatically remediates security findings without requiring analyst involvement
- D. It stores all security-related logs for seven years to meet compliance retention requirements



POP QUIZ: You run ``aws iam get-account-summary`` and see: {
"AccountMFAEnabled": 0, "AccountAccessKeysPresent": 0 } How
should this finding be classified, and what does it mean?

- A. Low severity — ``AccountAccessKeysPresent: 0`` means the account is secure
- B. Medium severity — MFA is recommended but not enforced on AWS root accounts
- C. High severity — the root account has no MFA, meaning anyone who obtains the root password has unrestricted access to the entire account
- D. Critical severity — ``AccountAccessKeysPresent: 0`` means the root user's access has already been revoked



POP QUIZ: You run ``aws iam get-account-summary`` and see: {
"AccountMFAEnabled": 0, "AccountAccessKeysPresent": 0 } How
should this finding be classified, and what does it mean?

- A. Low severity — ``AccountAccessKeysPresent: 0`` means the account is secure
- B. Medium severity — MFA is recommended but not enforced on AWS root accounts
- C. High severity — the root account has no MFA, meaning anyone who obtains the root password has unrestricted access to the entire account**
- D. Critical severity — ``AccountAccessKeysPresent: 0`` means the root user's access has already been revoked



POP QUIZ: A security group rule is changed from ``source: 0.0.0.0/0, port: 22`` to ``source: 203.0.113.42/32, port: 22``. What is the practical effect of this change?

- A. SSH is now fully disabled on the instance and no one can connect remotely
- B. The instance is now protected by a Web Application Firewall that inspects all SSH traffic
- C. SSH access is now restricted to a single IP address, eliminating exposure to automated internet scanners and unauthorized remote users
- D. The change has no effect on existing connections and only applies to new connection attempts from previously blocked addresses



POP QUIZ: A security group rule is changed from `source: 0.0.0.0/0, port: 22` to `source: 203.0.113.42/32, port: 22`. What is the practical effect of this change?

- A. SSH is now fully disabled on the instance and no one can connect remotely
- B. The instance is now protected by a Web Application Firewall that inspects all SSH traffic
- C. SSH access is now restricted to a single IP address, eliminating exposure to automated internet scanners and unauthorized remote users**
- D. The change has no effect on existing connections and only applies to new connection attempts from previously blocked addresses



LAB 1.4 — AWS SECURITY POSTURE BASELINE

